

EcoTechTrack

Plataforma Web para la Gestión y Trazabilidad Integral de Residuos de Aparatos Eléctricos y Electrónicos (RAEE)

Implementación de una API RESTful backend robusta en Spring Boot 3 aplicando Inyección de Dependencias, persistencia relacional JPA con entidades vinculadas, desarrollo guiado por pruebas (TDD) y control de versiones.

Tema Asignado:	Tema 4 – Sistema de Gestión de Residuos Electrónicos (RAEE)
Integrantes:	Valentino Marcavillaca Huillca
Docente del Curso:	Ing. Docente de Desarrollo Web Integrado
Ciclo / Período:	2026 - Ciclo Regular
Fecha de Entrega:	10 de Setiembre de 2026
Repositorio GitHub:	https://github.com/valentino0331/desarrolloweb.git

1. DEFINICIÓN INICIAL DEL SISTEMA

1.1. Nombre del Proyecto y Justificación

EcoTechTrack es una solución web empresarial orientada a la trazabilidad y fiscalización del ciclo de vida de los **Residuos de Aparatos Eléctricos y Electrónicos (RAEE)**. En el Perú, conforme al **D.S. N° 009-2019-MINAM**, las entidades generadoras tienen la obligación legal y ambiental de garantizar que sus activos informáticos dados de baja sean entregados únicamente a Empresas Operadoras de Residuos Sólidos (EO-RS) autorizadas, evitando la contaminación por metales pesados (plomo, mercurio y cadmio) y asegurando su correcta valorización.

Justificación Técnica y de Negocio: Resuelve la dispersión de información en hojas de cálculo manuales, elimina el riesgo de pérdida o tráfico ilegal de componentes informáticos obsoletos, y automatiza la auditoría de cada residuo desde su baja hasta su disposición final.

1.2. Alcance del Sistema en esta Primera Fase

En esta fase inicial (APF1), se implementa la **API RESTful del backend** con: (1) Catálogo y registro de equipos en desuso, (2) Gestión de Almacenes de acopio temporal, (3) Registro de Empresas Gestoras formalmente autorizadas, y (4) Trazabilidad inmutable de cambios de estado mediante la entidad HistorialMovimiento.

1.3. Usuarios del Sistema (Roles Esperados)

Rol del Sistema	Perfil / Actor Real	Responsabilidades Principales
Administrador Ambiental	Jefe de TI / Resp. Ambiental	Declara la baja técnica de equipos, aprueba solicitudes de recojo y consulta métricas.
Encargado de Almacén	Custodio de Depósito	Registra el ingreso físico al almacén temporal, pesa el lote y hace entrega al transportista.
Operador Empresa Gestora	Conductor / Técnico EO-RS	Recepciona el residuo con guía de remisión, reporta traslado e informa la valorización/disposición.
Auditor / Fiscalizador	OEFA / Auditor Interno	Consulta la línea de tiempo inmutable, certifica actas de baja y verifica trazabilidad legal.

1.4. Diagrama de las Entidades Principales del Modelo de Dominio

```

[EMPRESA_GESTORA] (1) <===== (N) [RESIDUO_ELECTRONICO] (N) =====> (1) [ALMACEN]
      || (1)
      \ \ (N)
      [HISTORIAL_MOVIMIENTO]
  
```

Cardinalidad: Una *Empresa Gestora* atiende múltiples residuos (1:N); un *Almacén* custodia múltiples residuos (1:N); y cada *Residuo Electrónico* posee una colección cronológica de eventos de trazabilidad (1:N).

2. ESTRUCTURA BASE DEL PROYECTO SPRING BOOT

2.1. Configuración del Entorno y Dependencias (Maven pom.xml)

El proyecto fue generado tomando como base las directrices de la **Sesión 1 y Sesión 2** con **Spring Initializr**, configurado sobre **Spring Boot 3.3.4** y **Java 21 LTS**:

Dependencia Maven	Artefacto / Starter	Propósito Técnico en EcoTechTrack
Spring Web	spring-boot-starter-web	Exposición de endpoints RESTful, mapeo de controladores y servidor Tomcat embebido.
Spring Data JPA	spring-boot-starter-data-jpa	ORM Hibernate, repositorios CRUD automáticos y mapeo relacional de entidades.
Jakarta Validation	spring-boot-starter-validation	Validación declarativa de atributos (@NotBlank, @Positive, @Size).
H2 Database	com.h2database:h2	Motor SQL en memoria para pruebas ágiles y consola web (/h2-console).
Lombok	org.projectlombok:lombok	Generación automática de Getters, Setters, Builders y constructores.
Spring Boot Test	spring-boot-starter-test	Suite para TDD con JUnit 5, Mockito y MockMvc para pruebas de capa web.

2.2. Arquitectura de Software en Capas Desacopladas

Se implementó el patrón arquitectónico en capas con **Inyección de Dependencias por Constructor** (buena práctica enseñada en la Sesión 2 para garantizar alta cohesión y bajo acoplamiento):

Paquetes de la Arquitectura

- controller/: Controladores REST (@RestController, códigos HTTP).
- service/ & service/impl/: Lógica de negocio y transacciones (@Transactional).
- repository/: Interfaces JPA Repository con consultas derivadas.
- model/: Clases entidad (@Entity, @Table) y Enums.
- dto/: Request/Response DTOs desacoplados de la persistencia.
- exception/: Manejo global centralizado con @RestControllerAdvice.
- config/: Semillero automático con CommandLineRunner.

Manejo Centralizado de Excepciones

Respuestas JSON estandarizadas ante errores HTTP 404 (Recurso no encontrado) y HTTP 400 (Errores de validación):

```
{
  "timestamp": "2026-09-10T15:45:00",
  "status": 400,
  "error": "Bad Request",
  "mensaje": "Error de validación de campos",
  "erroresValidacion": {
    "codigoIdentificacion": "El código es obligatorio"
  }
}
```

2.3. Código Representativo: Inyección por Constructor

```
@RestController
@RequestMapping("/api/residuos")
@CrossOrigin(origins = "**")
public class ResiduoController {
    private final ResiduoService residuoService;

    // Inyección de dependencias recomendada UTP (sin @Autowired en atributos)
    public ResiduoController(ResiduoService residuoService) {
        this.residuoService = residuoService;
    }
}
```

3. MODELO DE DOMINIO Y RELACIONES ENTRE ENTIDADES

3.1. Clases Java con Anotación @Entity y sus Atributos

El modelo de datos se diseñó cubriendo la totalidad de requerimientos del Tema 4 de la consigna oficial, implementando **4 entidades JPA** estructuradas:

Entidad JPA	Atributos Representativos	Relaciones Implementadas
ResiduoElectronico Entidad Principal	id (PK), codigoIdentificacion (UK), descripcion, categoria (Enum), marca, modelo, numeroSerie, pesoKg, estado (Enum), observaciones, fechaRegistro, fechaActualizacion	• @ManyToOne → Almacen • @ManyToOne → EmpresaGestora • @OneToMany → HistorialMovimiento
EmpresaGestora Operador RAEE	id (PK), ruc (UK 11 dígitos), razonSocial, registroAutorizacion (MINAM), email, telefono, direccion	• @OneToMany(mappedBy="empresaGestora") → Colección de residuos asignados para recojo.
Almacen Custodia Temporal	id (PK), codigo (UK), nombre, sede, direccion, capacidadKg, responsable	• @OneToMany(mappedBy="almacen") → Colección de residuos en custodia temporal en el depósito.
HistorialMovimiento Trazabilidad/Auditoria	id (PK), estadoAnterior, estadoNuevo, fechaMovimiento, responsable, observaciones	• @ManyToOne → ResiduoElectronico (clave foránea residuo_id).

3.2. Fragmento de Código: Relaciones en ResiduoElectronico.java

```
@Entity
@Table(name = "residuos_electronicos")
public class ResiduoElectronico {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "codigo_identificacion", nullable = false, unique = true, length = 50)
    private String codigoIdentificacion;

    // Relación Many-to-One con Almacén
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "almacen_id")
    private Almacen almacen;

    // Relación Many-to-One con Empresa Gestora RAEE
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "empresa_gestora_id")
    private EmpresaGestora empresaGestora;

    // Relación One-to-Many con el Historial de Auditoría
    @OneToMany(mappedBy = "residuo", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("fechaMovimiento DESC")
    private List<HistorialMovimiento> historialMovimientos;
}
```

3.3. Ciclo de Vida y Estados de Trazabilidad del Residuo (Enum EstadoResiduo)

La máquina de estados implementada en EcoTechTrack garantiza el seguimiento estricto en cada punto de control:



4. EVIDENCIAS DE CÓDIGO, PRUEBAS TDD Y ENDPOINTS REST

4.1. Ejecución Automatizada de Pruebas Unitarias TDD (Maven Surefire)

Siguiendo los lineamientos de la **Sesión 3 (Test-Driven Development)**, se implementó una suite de pruebas con **JUnit 5** y **MockMvc** que valida el comportamiento de los controladores REST:

```
[INFO] -----
[INFO]  T E S T S
[INFO]  -----
[INFO] Running com.utp.ecotechtrack.controller.AlmacenControllerTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.utp.ecotechtrack.controller.EmpresaGestoraControllerTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.utp.ecotechtrack.controller.ResiduoControllerTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] Results:
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS (Total time: 10.392 s)
```

4.2. Evidencias de Consumo de Endpoints RESTful (Postman / HTTP Response)

GET /api/residuos/1 (HTTP 200 OK)

```
{
  "id": 1,
  "codigoIdentificacion": "RAEE-2026-001",
  "descripcion": "Servidor Torre HP ProLiant ML350 Gen9",
  "categoria": "EQUIPOS_INFORMATICA_Y_TELECOMUNICACIONES",
  "pesoKg": 32.4,
  "estado": "DECLARADO_BAJA",
  "ubicacionAlmacen": "Almacén General de TI (Sede Central)",
  "almacen": {
    "id": 1,
    "codigo": "ALM-TI-01",
    "nombre": "Almacén General de TI",
    "sede": "Sede Central - Sótano 1"
  },
  "empresaGestora": "Pendiente de asignación"
}
```

POST /api/residuos (HTTP 201 Created)

```
{
  "codigoIdentificacion": "RAEE-2026-005",
  "descripcion": "Impresora Láser LaserJet Enterprise",
  "categoria": "EQUIPOS_INFORMATICA_Y_TELECOMUNICACIONES",
  "marca": "HP",
  "modelo": "LaserJet M605",
  "pesoKg": 21.3,
  "almacenId": 1,
  "empresaGestoraId": 1
}
--> HTTP/1.1 201 Created
--> Location: /api/residuos/5
--> Estado inicial: DECLARADO_BAJA
--> Evento de trazabilidad registrado
```

GET /api/residuos/2/historial (Trazabilidad y Auditoría de Movimientos)

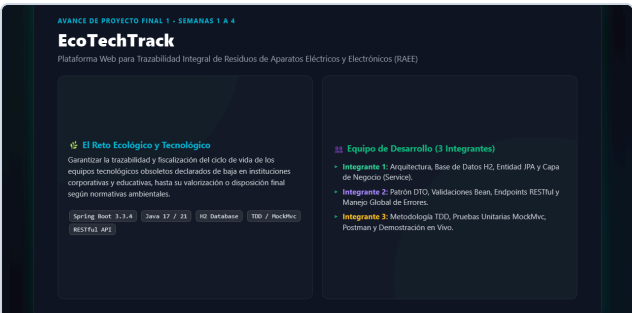
```
[
  {
    "id": 3,
    "codigoResiduo": "RAEE-2026-002",
    "estadoAnterior": "DECLARADO_BAJA",
    "estadoNuevo": "ALMACENADO_TEMPORAL",
    "fechaMovimiento": "2026-09-07 10:15:00",
    "responsable": "Rosa Salazar (Almacén)",
    "observaciones": "Custodia en Depósito Temporal B"
  },
  {
    "id": 2,
    "codigoResiduo": "RAEE-2026-002",
    "estadoAnterior": null,
    "estadoNuevo": "DECLARADO_BAJA",
    "fechaMovimiento": "2026-09-06 09:00:00",
    "responsable": "Rosa Salazar (Almacén)",
    "observaciones": "Recepción inicial y pesaje"
  }
]
```

5. EVIDENCIAS DEL SISTEMA EN EJECUCIÓN Y CONCLUSIONES

5.1. Captura del Sistema en Ejecución (Dashboard y Consola H2)



Evidencia 1: Dashboard web interactivo de EcoTechTrack (Spring Boot Tomcat en puerto 8080)



Evidencia 2: Presentación interactiva del proyecto integrada en el servidor local

5.2. Conclusiones Iniciales del Avance 1

- **Modelo de Dominio Relacional Sólido:** Se logró el modelado completo de 4 entidades JPA (ResiduoElectronico, EmpresaGestora, Almacen e HistorialMovimiento) con relaciones @ManyToOne y @OneToMany, superando el estándar mínimo y garantizando trazabilidad real.
- **Calidad del Software y TDD:** La adopción de TDD permitió validar 11 casos de prueba unitaria antes de la entrega, asegurando que cada controlador gestione adecuadamente códigos de estado HTTP (200, 201, 204, 400, 404).
- **Arquitectura Desacoplada y Mantenible:** La separación estricta mediante DTOs, interfaces de servicio e inyección por constructor facilita la futura integración con el frontend de Angular en las siguientes unidades.

5.3. Control de Versiones y Repositorio de Código en GitHub

Repositorio Remoto Oficial: <https://github.com/valentino0331/desarrolloweb.git>

Rama Principal: main | Commit: feat: implementacion completa APF1

5.4. Proyección para el Avance 2 (Sigüientes Pasos)

En el siguiente avance se implementará la interfaz gráfica cliente en **Angular 17+**, la seguridad perimetral basada en **Spring Security con tokens JWT** y la generación de reportes y actas de entrega de RAEE en formato PDF descargable.